Unity FSM 유한 상태 머신 활용

지원자: 엄태준

개발기간: 2/26 ~ 3/7, 2026 (약 1.5주)

영상: https://www.youtube.com/watch?v=PqhtsmqGaDc

GitHub: https://github.com/Seopyo27/Raze

Unity 6000.3.7f1

목차

1. 개요
   개발 동기, 개발 목표, 게임 설명
2. 상태 머신
   유한 상태 정의, 상태 패턴 적용
3. 게임 시스템
   스탯, 자동 점프
4. 애니메이션 및 연출
   애니메이션, 이벤트 기반 연출, 카메라 흔들림

1. 개요

개발 동기

- 게임 개발 중, 캐릭터 상태를 Switch문으로 분기하다가 한계를 경험하고 개발이 중단되었습니다.

- 상태가 늘어나면서 코드가 길어지고 분기 판단에 사용되는 플래그가 폭발적으로 증가했기 때문입니다.

- 특히 가장 큰 어려움은 이동하면서 공격, 점프하면서 공격, 점프하면서 이동 등 중첩 상태를 관리하는 것이었습니다.

- 따라서 다음과 같은 목표를 가지고 개발을 시작했습니다.

개발 목표

- 유한 상태 정의, 상태 패턴(State Pattern) 적용, 캐릭터 행동 로직 모듈화
- 새로운 상태를 쉽게 추가할 수 있는 유연한 상태 구조 설계
- 확장 가능한 데이터 기반 설계
- 다음과 같은 설계 패턴을 적용할 수 있는 게임 프로토타입 개발

게임 설명

- 게임 이름: Raze

- 게임 장르: 액션 RPG

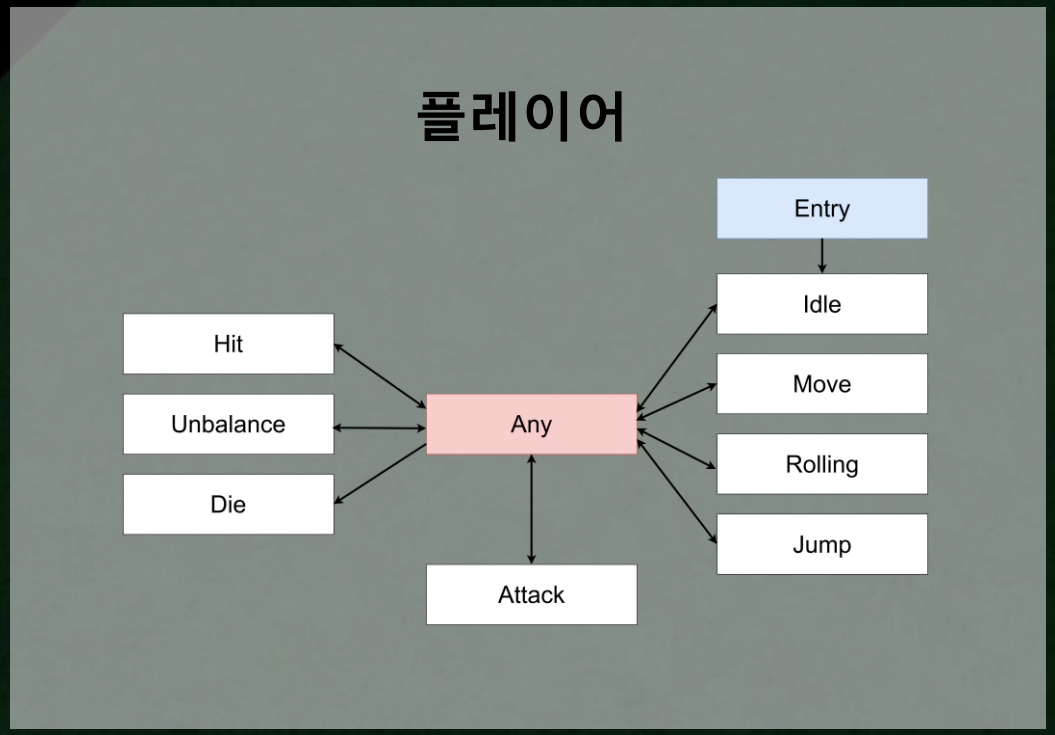
- 플랫폼: PC

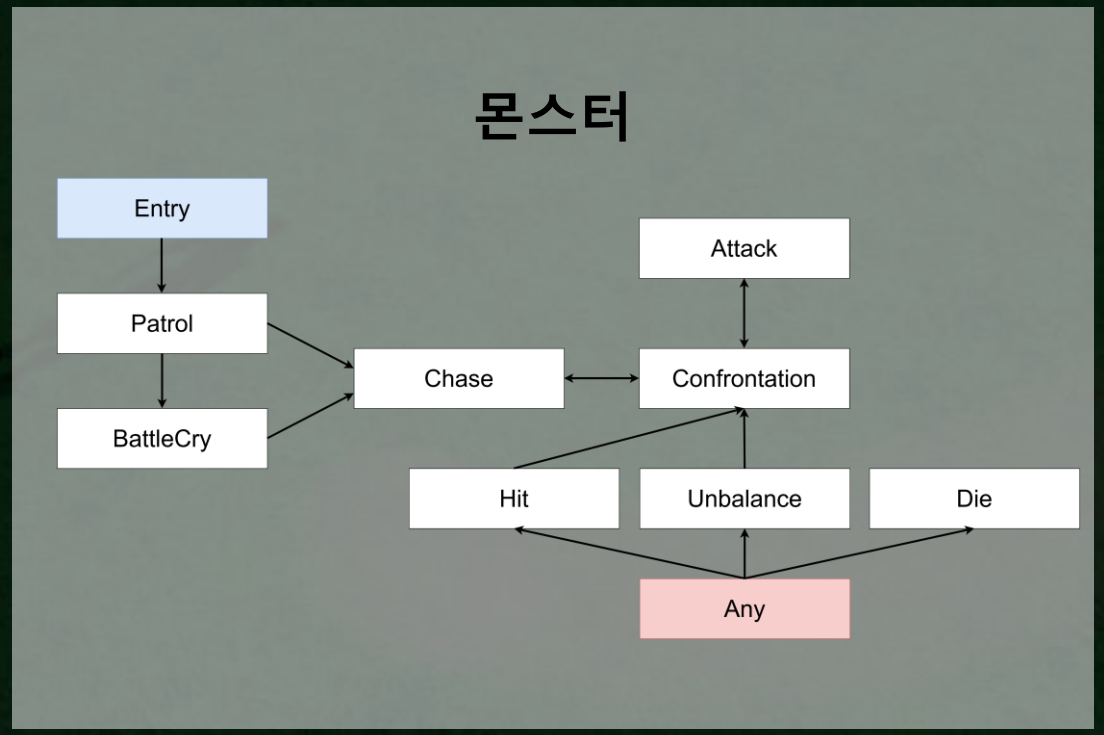
- 설명: 액션감, 타격감을 바탕으로 적을 물리치고 성장하는 게임입니다. 스태미나, 밸런스 등 전투에 사용되는 스탯이 존재하며 이를 관리하는 것이 중요합니다.

2. 상태머신

유한 상태 정의

- 플레이어와 적의 유한 상태를 정의했습니다.

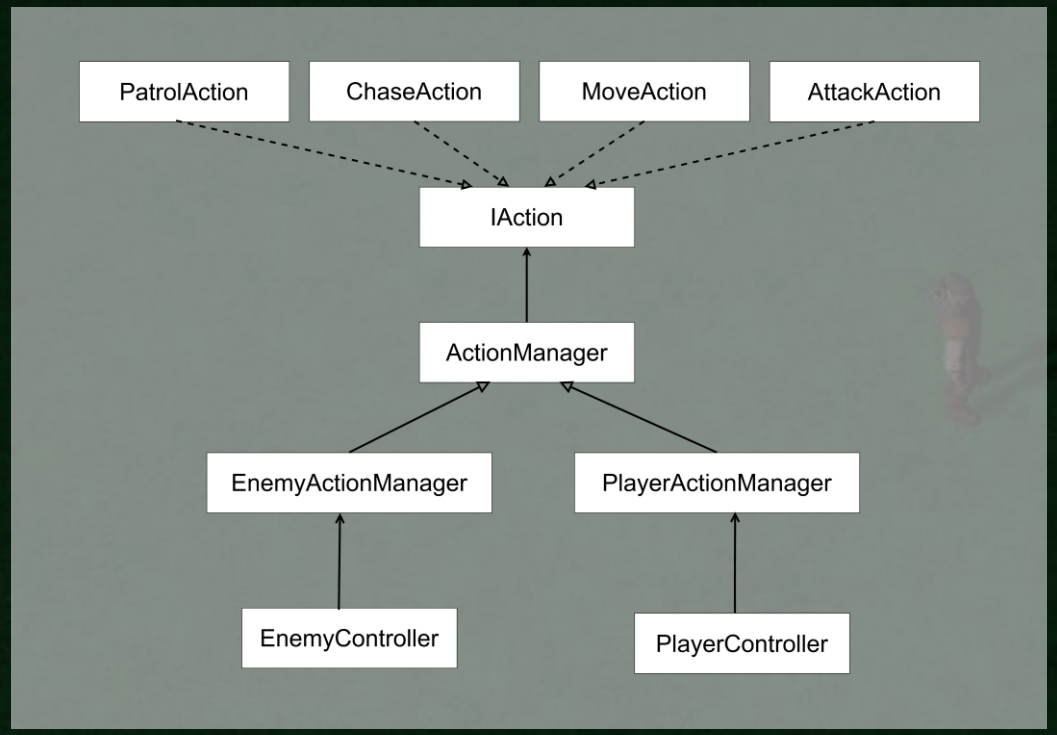




- 플레이어는 사용자 입력에 의해 상태 전이가 동적으로 발생합니다.

- 몬스터는 AI 로직에 의해 [정찰->추격->대치]와 같은 비교적 명확한 흐름을 따릅니다.

상태 패턴 적용

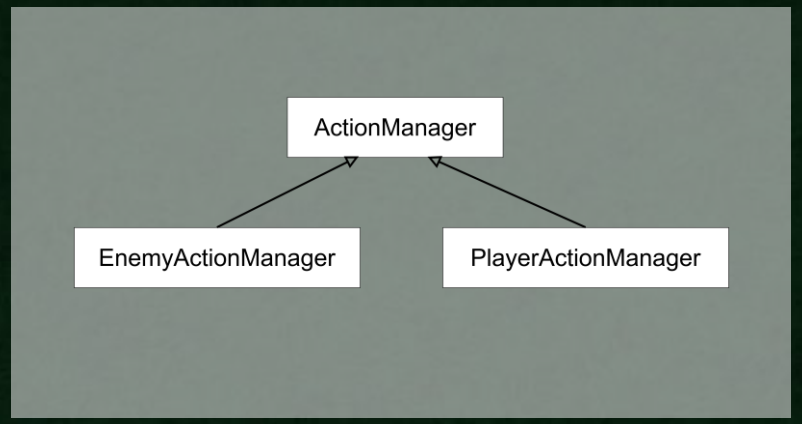



```
namespace MP1.Actions.Core
{
    1 reference
    public interface IAction
    {
        19 references
        public void Enter();
        18 references
        public void Update();
        18 references
        public void Exit();
    }
}
```

- FSM 구조를 기반으로 각 행동을 독립적인 Action 클래스로 분리하고 ActionManager가 관리하는 구조로 설계했습니다.

- 각 Action은 IAction인터페이스를 구현하기 때문에 공통된 생명주기로 작동합니다.

상태 패턴 적용



- 각 개체의 ActionManager를 분리한 이유는 2가지 입니다.

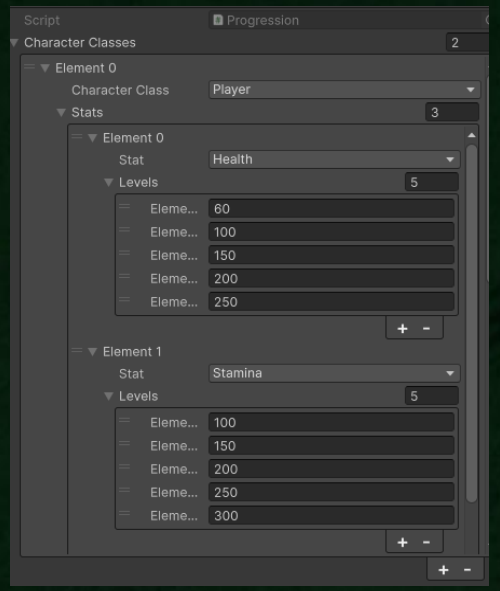
1. 각 개체가 사용하는 Action을 캐싱하기 위함입니다. 상태 전환 시 객체가 빈번하게 생성, 해체되지 않습니다. 하지만 각 상태들의 변수를 주기적으로 초기화하는 관리가 필요했습니다.

2. Action들의 중복 전이 조건 로직을 일괄적으로 관리하기 위함입니다. 특히 플레이어 같은 경우 조작에 따라 모든 Action으로 전이가 가능했기 때문에 중복 전이 조건 로직 관리가 필수였습니다.

3. 게임 시스템

스탯

```
[CreateAssetMenu(fileName = "Progression", menuName = "Stats/New Progression", order = 0)]

1 reference | Unity Script
public class Progression : ScriptableObject
{
    1 reference | Unity Serialized Field
    [SerializeField] ProgressionCharacterClass[] characterClasses = null;

    [System.Serializable]
    2 references
    class ProgressionCharacterClass
    {
        1 reference | Unity Serialized Field
        public characterClass characterClass;
        1 reference | Unity Serialized Field
        public ProgressionStat[] stats;

    }

    [System.Serializable]
    2 references
    class ProgressionStat
    {
        1 reference | Unity Serialized Field
        public Stat stat;
        1 reference | Unity Serialized Field
        public float[] levels;
    }
```



- 스탯은 ScriptableObject로 관리됩니다. 플레이어와 몬스터 모두 같은 방식으로 관리되며, Inspecter를 통해 새로운 스탯, 레벨, 적을 추가하기 쉽도록 설계했습니다.

체력

스태미너:
구르기, 공격 시 소모

밸런스:
피격 시 감소
전부 소진 시 불균형 상태 (스턴)

<스탯 설명>

스탯

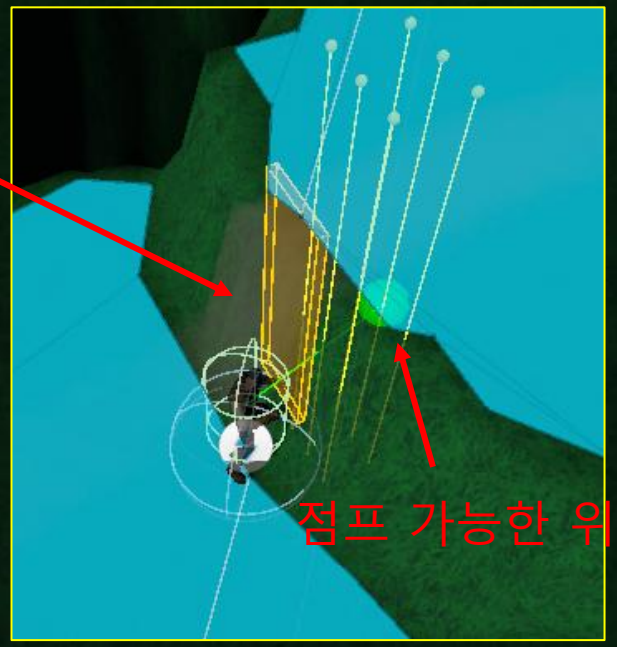
- 런타임 중 스탯 조회 최적화를 위해 Dictionary를 사용한 Lookup Table을 생성합니다. 최초 조회할 때 한번만 생성하며 스탯 조회 시간 복잡도를 O(1)로 최적화했습니다.

```
2 references
private void BuildLookup()
{
    if(lookupTable != null) return;

    lookupTable = new Dictionary<characterClass, Dictionary<Stat, float[]>>();

    foreach (ProgressionCharacterClass progressionClass in characterClasses)
    {
        var statLookupTable = new Dictionary<Stat, float[]>();

        foreach (ProgressionStat progressionStat in progressionClass.stats)
        {
            statLookupTable[progressionStat.stat] = progressionStat.levels;
        }

        lookupTable[progressionClass.characterClass] = statLookupTable;
    }
}
```

자동 점프

- 본 게임에는 수동 점프기능이 없습니다. 하지만 게임 내 수직적 레벨 디자인을 활용하기 위해, 언덕 이동 시 캐릭터가 자동으로 점프하도록 하는 시스템을 구축했습니다.

- Raycast, Boxcast를 사용해 언덕과 올라갈 수 있는 곳을 탐지합니다. NavMesh 기반으로 이동 가능 위치를 검증합니다. 가능한 후보군 중에서 캐릭터와 가장 가까운 거리로 최적 위치를 선택합니다.




```
private Vector3 GetBestClimbJumpPosition()
{
    Vector3 bestTarget = Vector3.zero;
    float closestDistance = float.MaxValue;

    Vector3 originCenter = transform.position + Vector3.up * climbJumpUpOffset + _player.forward * climbJumpForwardOffset;

    // 3 * 2 격자 스캔
    for (int x = -1; x <= 1; x++)
    {
        for (int z = 0; z <= 1; z++)
        {
            // 1. 각 레이 시작 지점 계산
            Vector3 scanOffset = (transform.right * x * (climbJumpRowGap / 2f)) + (_player.forward * z * (climbJumpColGap / 2f));
            Vector3 rayStartPosition = originCenter + scanOffset;

            // 2. 시작 지점에서 위에서 아래로 레이를 쏴서 바닥 충돌 확인
            if (!Physics.Raycast(rayStartPosition, Vector3.down, out RaycastHit hit, climbJumpUpOffset, _groundLayer)) continue;

            // 3. 이동 가능한 NavMesh 위인지 확인
            NavMeshHit navHit;
            if (!NavMesh.SamplePosition(hit.point, out navHit, 0.5f, NavMesh.AllAreas)) continue;

            // 4. 캐릭터가 점프할 수 있는 높이 인지 확인
            float heightDiff = navHit.position.y - transform.position.y;
            if (!(heightDiff >= climbJumpMinHeight && heightDiff <= climbJumpMaxHeight)) continue;

            // 5. 가장 가까운 곳인지 확인
            float dist = Vector3.Distance(transform.position, navHit.position);
            if (dist >= closestDistance) continue;

            closestDistance = dist;
            bestTarget = navHit.position;
            debugTargetPos = bestTarget;
        }
    }

    return bestTarget;
}
```

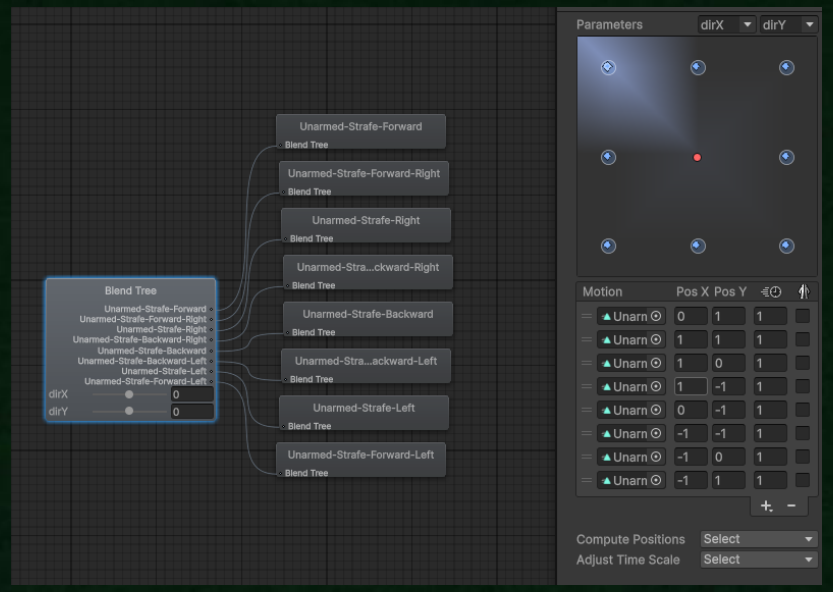
자동 점프

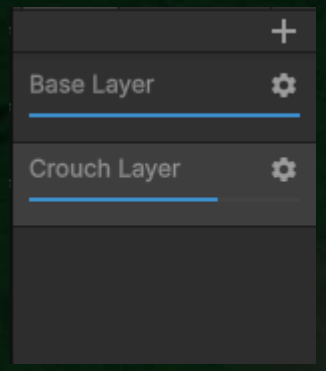
- 앞 낭떠러지를 뛰어 넘을 수 있는 점프도 가능합니다. 언덕을 오르는 점프 기능에서 센서 부분인 Raycast, Boxcast 부분이 달라집니다.

```
/// <summary>
/// 전방을 스캔하고 리프 점프 가능한 착지 지점 중 가장 가까운 NavMesh 위치 반환
/// </summary>
/// <returns>가장 가까운 리프 점프 목표 위치. 없으면 Vector3.zero</returns>
1 reference
private Vector3 GetBestLeapJumpPosition()
{
    Vector3 bestTarget = Vector3.zero;
    float closestDist = float.MaxValue;

    //전방 3곳 스캔
    for (int i = -1; i <= 1; i++)
    {
        // 1. 각 레이 시작 지점 계산
        Vector3 horizontalOffset = transform.right * (i * leapJumpCheckLineGap * 0.5f);
        Vector3 leapgroundLayStartPosition = transform.position + (transform.forward * leapJumpCheckForwardOffset) + horizontalOffset

        // 2. 시작 지점에서 위에서 아래로 레이를 쏴서 바닥 충돌 확인
        if (!Physics.Raycast(leapgroundLayStartPosition, Vector3.down, out RaycastHit hit, leapJumpCheckDepth, _groundLayer)) continu

        // 3. 이동 가능한 NavMesh 위인지 확인
        NavMeshHit navHit;
        if (!NavMesh.SamplePosition(hit.point, out navHit, 0.5f, NavMesh.AllAreas)) continue;

        // 4. 캐릭터가 점프할 수 있는 높이 인지 확인
        float heightDiff = Mathf.Abs(navHit.position.y - transform.position.y);

        if (heightDiff <= leapJumpCheckheightTolerance)
        {
            float d = Vector3.Distance(transform.position, navHit.position);
            if (d < closestDist)
            {
                closestDist = d;
                bestTarget = navHit.position;
            }
        }
    }

    return bestTarget;
}
```

4. 애니메이션 및 연출

애니메이션





- 적 대치 시 8방향 이동 애니메이션은 animator의 Blend Tree로 구현했습니다. 이동 방향에 따라 자연스러운 애니메이션 전환이 가능하고, 복잡한 상태 전이를 최소화 할 수 있습니다.

- 앉기 애니메이션은 레이어 가중치를 조절, 애니메이션 혼합 방식을 사용했습니다. 따라서 기존 이동 애니메이션을 활용해서 앉아서 이동 애니메이션을 구현했습니다.

이벤트 기반 연출

피격 시 연출

```
public class Health : MonoBehaviour
{
    2 references
    private CharacterStats _characterState;
    5 references | Unity Serialized Field
    [SerializeField] private float currentHP;
    4 references | Unity Serialized Field
    [SerializeField] private bool isDead;

    4 references
    public event Action OnHit;
    4 references
    public event Action OnDie;
```

```
public class Balance : MonoBehaviour {
    2 references
    private CharacterStats _characterState;
    9 references | Unity Serialized Field
    [SerializeField] private float _currentBA;
    2 references | Unity Serialized Field
    [SerializeField] private float _recoveryDelayTime = 0.5f;
    1 reference | Unity Serialized Field
    [SerializeField] private float _recoveryRate = 10f;
    5 references | Unity Serialized Field
    [SerializeField] private bool _isUnbalanced;

    2 references
    public event Action OnUnbalance;
```

불균형 상태 연출

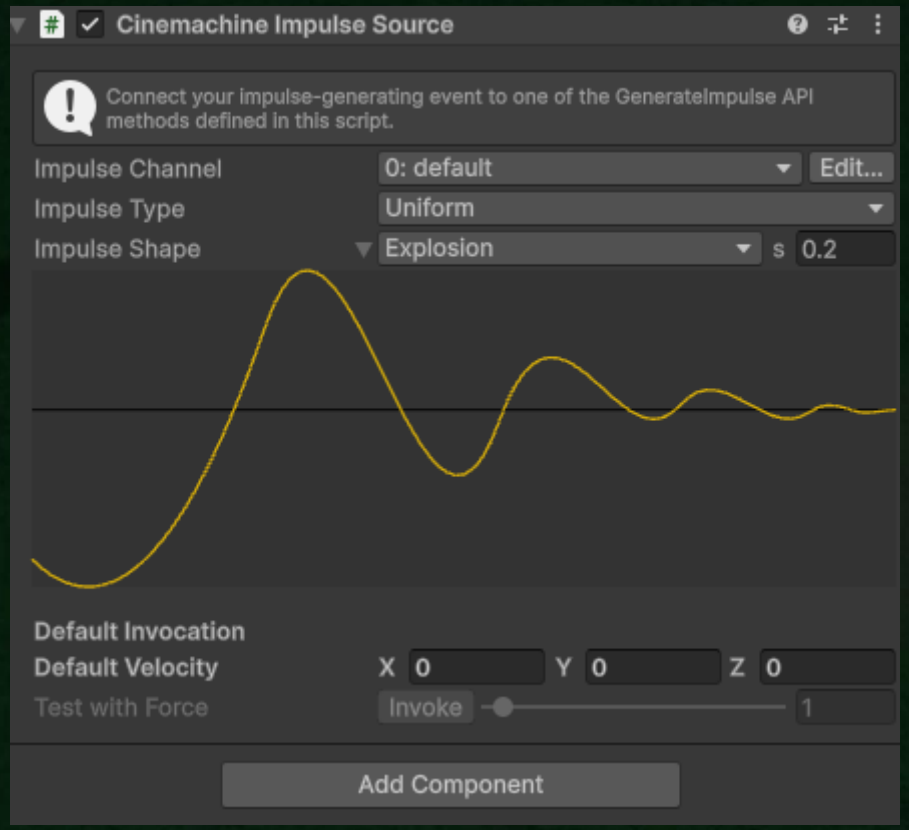
```
private CharacterStats _characterState;
8 references | Unity Serialized Field
[SerializeField] private float _currentSP;
2 references | Unity Serialized Field
[SerializeField] private float _recoveryDelayTime = 0.5f;
1 reference | Unity Serialized Field
[SerializeField] private float _recoveryRate = 10f;

2 references
public event Action OnNoStamina;
```

스태미너 부족 연출

- 각종 연출은 Event 구조를 활용해서 시스템 간 의존성을 낮추는 방식으로 설계했습니다. 따라서 새로운 연출을 추가할 때 이벤트 구독만으로 기능을 확장할 수 있습니다.

카메라 흔들림 연출

```
1 reference
private void ShakeCamera(int comboIndex)
{
    if (_player.ImpulseSource == null) return;

     Vector3 shakeDirection = _player.transform.forward;


    if (comboIndex < 3)
    {
        _player.ImpulseSource.GenerateImpulseWithVelocity(shakeDirection * 0.1f);
    }
    else
    {
        _player.ImpulseSource.GenerateImpulseWithVelocity(shakeDirection * 0.2f);
    }
}
```



- Cinemachine의 Cinemachine Impulse를 사용해 공격 시 카메라 흔들림을 구현했습니다. 공격 방향에 따라 흔들림의 방향이 달라지며, 공격 콤보 수가 커지면 흔들림의 강도가 강해집니다.

결론

- 유한 상태 정의, 상태 패턴 적용으로 모든 개체의 상태를 통제했습니다.

- 이러한 설계로 추후 상태들의 확장에도 유연하게 대처할 수 있습니다.

- 스탯, 무기, 적들의 시야 등의 데이터는 전부 ScriptableObject 기반 설계로 재사용성과 확장성을 향상시켰습니다.

감사합니다.

에셋

모델 & 애니메이션

- https://assetstore.unity.com/packages/3d/characters/humanoids/humans/free-modular-character-fantasy-rpg-human-male-228952

- https://assetstore.unity.com/packages/3d/characters/humanoids/fantasy/lowpoly-medieval-skeleton-free-low-poly-medieval-fantasy-series-181883

- https://assetstore.unity.com/packages/3d/animations/human-basic-motions-free-154271

- https://assetstore.unity.com/packages/3d/animations/rpg-character-mecanim-animation-pack-free-65284

사운드

- https://assetstore.unity.com/packages/audio/sound-fx/free-deadly-kombat-228835

연출

- https://assetstore.unity.com/packages/vfx/shaders/fullscreen-camera-effects/quirky-s-easy-animatable-vignette-177253